

Project Report — In-App View

Making the app the primary way to view the report, with Excel/PDF as exports — Contracts ERP

Audit finding from the project-report presentation · `GetProjectReport` → KendoReact Grid screen · 10 September 2026

Status: Draft for review · **Date:** 2026-09-10 · **Owner:** Emad Radwan

Trigger: During the project-report presentation, the client's **auditor** objected that the system must not depend on Excel as the *primary* way to view a report. The report has to be **viewable inside the application** — the way the payment voucher now is — with **Excel and PDF as "export / save-as" options**, not as the report itself.

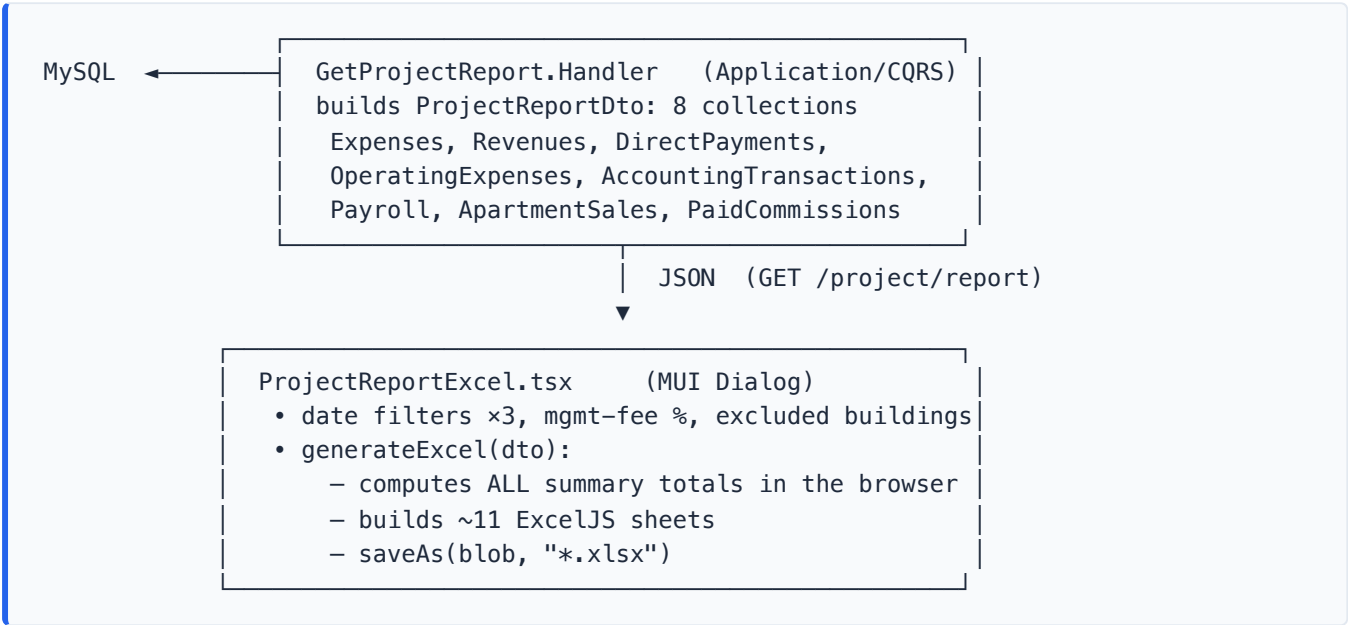
This plan covers only the **Project Report** (`GetProjectReport` → `client-app/.../Projects/report/ProjectReportExcel.tsx`). The same refactor applies 1:1 to the company-level report (`fetchCompanyReport` , `/project/companyReport`), which returns the identical `ProjectReportDto` .

1. This is not the same problem as the payment voucher

	Payment voucher (done)	Project report (this plan)
Shape	one fixed page, print layout	8 sections, hundreds of rows, analytical
What the viewer does	reads / prints it	filters, sorts, groups, traces a total to its rows
Right primary surface	a code-defined Telerik report	an in-app screen built on the KendoReact Grid (v16)
Telerik Reporting's role	<i>is</i> the report	one of the export formats (PDF), secondary

A rendered PDF or spreadsheet is static — it can't be the "layer you work in" for a report this size. The KendoReact Grid gives sorting, filtering, grouping, column reordering and its own Excel/PDF export **for free**, and the data already arrives as JSON.

2. Current architecture (as-is)



Files involved today

File	Role
<code>Application/Projects/GetProjectReport.cs</code>	the query + handler — one DB read per section, all calculation server-side
<code>Application/Projects/ProjectReportDto.cs</code>	8 collection types (<code>ProjectExpenseRecord</code> , <code>ProjectRevenueRecord</code> , <code>ProjectCommissionPaymentRecord</code> , shared <code>PaymentRecord</code> , <code>SalesRequestOrApartmentRecord</code>)
<code>client-app/src/app/store/apis/projectsApi.ts</code>	<code>fetchProjectReport</code> query (<code>useLazyFetchProjectReportQuery</code>); FE <code>ProjectReportDto</code> interface
<code>client-app/src/features/Projects/report/ProjectReportExcel.tsx</code>	the entire user-facing feature — a dialog that produces an <code>.xlsx</code>
<code>client-app/src/features/Projects/dashboard/ProjectsList.tsx:280</code>	opens <code><ProjectReportExcel></code> from a per-row action, gated by role <code>RunProjectReport</code>

3. The gap — precisely

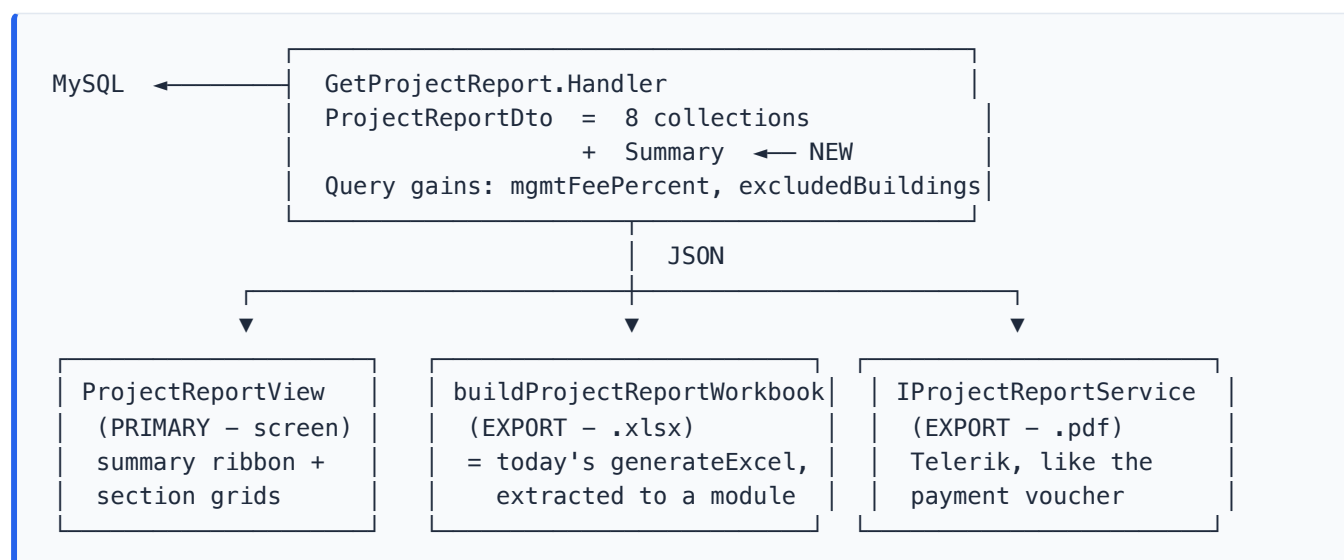
1. **There is no in-app view.** The only way to see the report is to download a spreadsheet and open it. That is exactly what the auditor rejected.
2. **The summary numbers are computed in the browser.** `ProjectReportExcel.tsx:223–296`
(`sumSection` / `sumLine`) derives every total client-side — certificate/direct/transaction/ payroll/operating expense subtotals and grand total, revenue scheduled/collected/outstanding, maintenance-deposit split,

unit-sales totals, commissions paid/pending, and the management-fee block (`mgmtBase` , `mgmtFee` , `mgmtNet`). If a second surface (a screen, a PDF) computes its own totals, the two **will drift** — and there is already a known rounding rule the reports must respect (`PAYROL_PAYMENT` nets are rounded per-invoice to 2 dp *then* summed, never raw-summed).

3. **Management-fee inputs live only in the dialog.** `mgmtFeePercent` (default 12) and `excludedBuildings` (pre-selects A1/A2) are dialog state, fed by `useFetchProjectBuildingsQuery` . Any new surface needs the same two inputs — so they belong in the query, not one screen's state.

4. Target architecture

One calculation layer. Three surfaces, all reading the same DTO.



Principle: no number exists only in a spreadsheet formula. Every total is computed once, in `GetProjectReport` , shown on the screen, and re-used verbatim by both exports.

5. The in-app screen — `ProjectReportView`

A full screen (not a dialog), opened from the same `RunProjectReport` action on `ProjectsList` .

5.1 Filter bar (top, sticky)

Everything the `Query` already takes, plus the two mgmt-fee inputs pulled out of the old dialog:

- Project (fixed when opened from a row; a picker for the company report)
- Expenses period · Revenues period · Sales period — each a date range + an "All data" toggle
- Management fee % (`10` / `11` / `12` / `12.5` / `13` / `14` / `15` , default 12)
- Excluded buildings (multi-select from `fetchProjectBuildings` , pre-checks A1/A2)
- **Actions:** `Refresh` · `Export` ▾ (Excel · PDF)

Changing a filter re-runs `useLazyFetchProjectReportQuery` (one call, returns the whole DTO).

5.2 Summary ribbon

Cards / a compact table driven **entirely** by `dto.summary` — the same sections the Excel summary sheet has today:

Group	Lines
المصاريف	المستخلصات · الدفعات المباشرة · قيود محاسبية · رواتب المشروع · المصاريف التشغيلية · إجمالي مصاريف المشروع
الإيرادات	الإيراد المتفق عليه · المحصل · المتبقي
وديعة الصيانة	الإجمالي · المحصل · المتبقي
مبيعات الوحدات	عدد الوحدات المباعة · إجمالي القيمة · إجمالي المقدمات المحصلة · عدد الوحدات المتاحة
العمولات	عدد الدفعات · المدفوعة · المستحقة · الإجمالي
مبلغ الإدارة	المصاريف التشغيلية · الصافي المتبقي - (x%) الأساس (± المستبعد) · النسبة
الصافي	المحصل - مصاريف المشروع · بعد خصم العمولات المدفوعة

5.3 Section grids

One KendoReact `Grid` (v16) per DTO collection, as tabs or an accordion. Column sets **mirror the current Excel sheets** so nothing is lost versus the spreadsheet:

Tab (Arabic)	DTO collection	Notes
المصاريف - المستخلصات	<code>expenses</code>	~15 cols; group by certificate; footer = net total
الدفعات المباشرة	<code>directPayments</code>	shared <code>PaymentRecord</code> columns
قيود محاسبية	<code>accountingTransactions</code>	<code>PaymentRecord</code>
رواتب المشروع	<code>payroll</code>	<code>PaymentRecord</code> ; per-invoice 2 dp rounding already applied server-side
المصاريف التشغيلية	<code>operatingExpenses</code>	<code>PaymentRecord</code>
الإيرادات	<code>revenues</code>	due-status buckets → filterable columns
وديعة الصيانة	<code>revenues</code> filtered to maintenance	same grid component, pre-filter
مبيعات الوحدات	<code>apartmentSales</code>	sold + available
العمولات المدفوعة	<code>paidCommissions</code>	paid vs. still-owed split (<code>isPaid</code>)

Each grid: sort, multi-column filter, group, column show/hide, and per-grid `Export to Excel` (KendoReact built-in) for the "just this section" case.

Empty sections render a muted "لا توجد بيانات للفترة المحددة" instead of an empty tab.

6. Backend changes

6.1 ProjectReportDto — add Summary (additive, no breakage)

```
public class ProjectReportDto
{
    // ... 8 existing collections, unchanged ...
    public ProjectReportSummaryDto Summary { get; set; } = new();    // NEW
}

public class ProjectReportSummaryDto
{
    // المصاريف
    public decimal CertificateExpenses { get; set; }
    public decimal DirectPayments { get; set; }
    public decimal AccountingTransactions { get; set; }
    public decimal ProjectPayroll { get; set; }                // per-invoice 2dp rounded then
summed
    public decimal OperatingExpenses { get; set; }
    public decimal TotalProjectExpenses { get; set; }

    // الإيرادات / وديعة الصيانة
    public decimal RevenueScheduled { get; set; }
    public decimal RevenueCollected { get; set; }
    public decimal RevenueOutstanding { get; set; }
    public decimal MaintenanceScheduled { get; set; }
    public decimal MaintenanceCollected { get; set; }
    public decimal MaintenanceOutstanding { get; set; }

    // مبيعات الوحدات
    public int UnitsSold { get; set; }
    public decimal UnitsSoldValue { get; set; }
    public decimal UnitsAdvanceCollected { get; set; }
    public int UnitsAvailable { get; set; }

    // العمولات
    public int CommissionPaymentCount { get; set; }
    public decimal CommissionsPaid { get; set; }
    public decimal CommissionsPending { get; set; }

    // مبلغ الإدارة
    public decimal MgmtFeeBase { get; set; }
    public decimal MgmtFeePercent { get; set; }
    public decimal MgmtFee { get; set; }
    public decimal MgmtFeeNet { get; set; }
    public IReadOnlyList<string> MgmtExcludedBuildings { get; set; } = Array.Empty<string>
();

    // الصافي
    public decimal NetAfterExpenses { get; set; }                // collected – total expenses
    public decimal NetAfterPaidCommissions { get; set; }        // – commissions paid
}
```

The handler computes these from the collections it already builds — this is a **lift-and-shift** of `ProjectReportExcel.tsx:223–296` into C#, nothing new is calculated.

6.2 `GetProjectReport.Query` — two new params

```
public decimal MgmtFeePercent { get; set; } = 12m;
public List<string> ExcludedBuildings { get; set; } = new(); // default: server pre-
fills A1/A2 if present
```

Needed so `Summary.MgmtFee*` can be computed server-side. Wire them through `fetchProjectReport` / `fetchCompanyReport` in `projectsApi.ts`.

6.3 (Optional) `IPProjectReportService` for server-side PDF/XLSX

Mirrors `IPaymentVoucherReportService` — `Render(ProjectReportDto dto, string format)`. Only needed if we want the PDF produced on the server rather than from the Grid. Decide in §7.

7. PDF export — options

Option	Effort	Fidelity	Notes
A. KendoReact Grid <code>PDFExport</code> per section + a summary page	Low	Fair	Ships with v16; one file, section-per-page; quickest path to "there is a PDF button"
B. Telerik <code>ProjectReport</code> (code-defined) bound to the DTO, rendered by <code>IPProjectReportService</code>	Medium	High	Same pattern as the payment voucher; print-quality summary + section tables; server-side
C. Headless-Chrome print of the screen	Low–Med	Good	Reuses the exact on-screen layout; needs a print stylesheet

Recommendation: ship **A** with the first release (low cost, satisfies "PDF is an export option"), then add **B** if the auditor or client wants a formal, paginated PDF. Excel export is **not** in this table — it stays as today's hand-built workbook (§8.2).

8. Frontend changes

8.1 New: `features/Projects/report/ProjectReportView.tsx`

The screen from §5. Uses `useLazyFetchProjectReportQuery`, KendoReact `Grid` per section, a summary ribbon component, and the filter bar. Follows the repo's Menu-Exit pattern for list↔view switching (`client-app/CLAUDE.md`).

8.2 Refactor: `ProjectReportExcel.tsx` → a pure module

Extract `generateExcel(dto)` into `features/Projects/report/buildProjectReportWorkbook.ts`:

```
export async function buildProjectReportWorkbook(
  dto: ProjectReportDto,
  opts: { projectName: string; periods: {...} }
): Promise<ArrayBuffer>
```

- No React, no hooks — just DTO in, `.xlsx` buffer out.
- Delete the browser-side summing; read totals from `dto.summary`.
- The existing dialog keeps working during the transition (it just calls the new function).
- `ProjectReportView` 's "Export → Excel" button calls the same function.

8.3 Switch the entry point

`ProjectsList.tsx:280` — replace `<ProjectReportExcel open=...>` with navigation to `ProjectReportView`. Once parity is confirmed, delete `ProjectReportExcel.tsx` (the dialog shell; the workbook logic lives on in the module).

9. Migration path (low risk, incremental)

Step	Change	Ships behind
1	Backend: <code>ProjectReportSummaryDto</code> + handler fills it (lift from the TSX summary)	nothing — additive
2	Backend: <code>Query.MgmtFeePercent</code> + <code>ExcludedBuildings</code> ; wire through <code>projectsApi.ts</code>	nothing — additive
3	FE: extract <code>buildProjectReportWorkbook.ts</code> ; old dialog calls it; totals now from <code>dto.summary</code>	old dialog (unchanged UX)
4	FE: <code>ProjectReportView</code> — filter bar + summary ribbon + one section grid (المستخلصات)	a hidden route / feature flag
5	FE: add remaining section grids one at a time, each checked against its Excel sheet	same
6	FE: "Export → Excel" (reuses step 3) + "Export → PDF" (option A) on the screen	same
7	Flip <code>ProjectsList</code> action to open the screen; keep the dialog reachable for one release	—
8	Delete <code>ProjectReportExcel.tsx</code> dialog shell	—
9	(Optional) <code>IProjectReportService</code> + Telerik <code>ProjectReport</code> for a formal PDF	—

Every step is independently shippable and reversible. Steps 1–3 have **zero** user-visible change.

10. Effort & sequencing

Block	Rough size
Steps 1–2 (backend summary + params)	~0.5–1 day — mechanical port of existing JS math
Step 3 (extract workbook module)	~0.5 day

Block	Rough size
Steps 4–5 (screen + 9 grids)	~3–5 days — the bulk; grid column defs mirror existing sheets
Step 6 (export buttons)	~0.5 day (Excel reuse) + ~0.5 day (Grid PDF)
Steps 7–8 (cutover + cleanup)	~0.5 day
Step 9 (Telerik PDF, optional)	~1–2 days

11. Risks & watch-items

#	Item	Mitigation
R1	Freshly changed code. The mgmt-fee block, maintenance-deposit split and paid-commissions sheet were coded 2026-09-10 and are still in the working tree, not yet built/verified.	Land and verify that batch first ; build this plan on top of it, not in parallel
R2	Summary drift between screen, Excel, PDF	Single Summary DTO is the whole point — no surface recomputes totals
R3	Payroll rounding (PAYROL_PAYMENT per-invoice 2 dp → sum)	Already handled in the handler; the ported Summary must call the same path, never raw-sum payroll
R4	RTL / Arabic in KendoReact Grid	v16 Grid supports dir="rtl" ; the app already runs RTL grids elsewhere
R5	Column parity — a stakeholder misses a column that was in the sheet	Step 5 checks each grid against its Excel sheet before enabling
R6	RunProjectReport role	screen sits behind the same <Can perform="RunProjectReport"> / route guard as today
R7	Company report (/project/companyReport) diverges	build ProjectReportView to take the DTO + a mode prop ('project' or 'company') from the start
R8	Large projects — hundreds of expense rows	Grid virtual scrolling; the DTO is already returned whole today, so payload size is unchanged

12. What to tell the auditor

- The **application now displays** the project's financial position — summary and every supporting row — with live filtering and sorting.
- **Excel and PDF are export actions**, generated on demand from the same server query that feeds the screen; they are a snapshot of what the system shows, not a separate calculation.
- No figure in the report originates in a spreadsheet formula. Every total is computed in **GetProjectReport** on the server and is traceable on screen to the transactions behind it.

13. Decisions needed

1. PDF export: start with **option A** (KendoReact Grid PDF) and defer Telerik (**B**)? — recommended.
2. Screen layout for the sections: **tabs** or a single scrolling **accordion**? — tabs recommended for 9 sections.
3. Do we cut the company-level report (`fetchCompanyReport`) over in the same effort, or fast-follow?
4. Confirm the 2026-09-10 enhancements batch is verified before step 1 begins.